Coding Agents

Last modified: 2026-11-10 12:34:03 (PST)

We recommend working with **AI coding agents** to help you code.

1 What Is a Language Model?

A **large language model** (LLM) is a statistical model trained to predict the next token in a sequence of text, based on patterns learned from enormous amounts of text during training. That single capability — predicting what comes next — turns out to be enough to write code, answer questions, and hold a conversation, once the model is large enough and trained on enough data.

A base model trained only to predict text is not yet a helpful assistant. A further training step, often reinforcement learning from human feedback, teaches the model to follow instructions, answer as a helpful assistant, and refuse harmful requests, rather than simply continuing whatever text it is given. This is the step that turns a raw language model into something like Claude or ChatGPT.

A model call is stateless: given the same input, it has no memory of any previous call. Every capability the rest of this chapter describes — holding a conversation, using tools, running autonomously as an agent — is scaffolding built on top of that one stateless function. The harness supplies the memory, the tools, and the control flow; the model only ever predicts what token comes next.

For a from-scratch walk through that stack, see Stanford's CS336: Language Modeling from Scratch. The course has students implement:

- data collection and cleaning for pre-training
- transformer construction
- training
- evaluation

Lecture recordings are public. It is an implementation-heavy treatment of how a language model is built, not a product tutorial for coding agents.

2 What are AI coding agents?

AI coding agents are AI agents specialized for coding. They differ from other AI coding tools in important ways:

Compared to inline coding assistants (like traditional autocomplete), coding agents work autonomously rather than providing suggestions as you type. They can navigate entire codebases, execute commands, and complete multi-step tasks without constant human guidance.

Compared to AI chatbots (like ChatGPT or Claude), coding agents don't just generate code snippets in conversation—they actively interact with your development environment. While chatbots require you to copy code from a chat window and manually integrate it into your project, coding agents directly read your codebase, make changes to files, run tests and build commands, and create pull requests with their proposed changes. Chatbots are conversational assistants; coding agents are autonomous development tools.

Coding agents are autonomous software programs that can:

- **Understand and execute complex tasks**: Coding agents can interpret natural language instructions and break them down into actionable development tasks
- **Navigate and modify codebases**: They can read, understand, and edit multiple files across a repository to implement features or fix bugs
- **Run tools and commands**: Coding agents can execute build commands, run tests, use linters, and interact with development tools
- **Make decisions autonomously**: They can plan their approach, make technical decisions, and adjust their strategy based on results
- **Work iteratively**: Coding agents can test their changes, identify issues, and refine their solutions through multiple iterations
- **Create comprehensive solutions**: They can implement complete features that span multiple files, including code, tests, and documentation

Coding agents operate in isolated environments where they can safely experiment and validate changes before proposing them. This allows them to work more independently than inline coding assistants, which require step-by-step human direction. The agent workflow typically involves analyzing requirements, planning an implementation, making changes, testing those changes, and creating a pull request with the results.

While coding agents can handle substantial development tasks, they still require human oversight and review. The human developer remains responsible for:

- Reviewing the agent's work
- Ensuring the solution meets requirements
- Verifying code quality and security
- Making the final decision to merge changes

3 Coding-Agent Platforms

Coding-agent platforms differ more in where they run and how they return work than in their chat interfaces. Choose a platform by matching its execution model to the task and your repository's security requirements.

Common coding-agent platforms

The following catalog is a starting point rather than an endorsement:

Platform	Primary surface	Typical repository workflow
GitHub Copilot coding agent	GitHub issues and pull requests	Assign an issue; review the resulting pull request
OpenAI Codex	Cloud tasks, app, and command line	Delegate a task in an isolated environment or work locally
Google Jules (Google 2026d)	Cloud coding agent	Connect a repository and review the proposed changes
Google Antigravity	Agentic development platform	Coordinate coding tasks in a managed development workspace
Claude Code	Terminal, IDE, and cloud	Work interactively or delegate work that returns a pull request
Gemini CLI	Command line	Inspect, edit, and test the current local checkout
Cursor (Cursor 2026)	IDE and cloud agents	Edit interactively with path-scoped `.cursor/rules` context, or hand a task to a cloud agent that pushes a branch
Aider	Command line	Pair locally with explicit files and Git commits
GitKraken Kepler	Agentic development environment (desktop)	Start a Task from an issue, PR, or idea; run multiple agents in parallel isolated worktrees via reusable Actions; review per-branch diffs and open PRs

Platform	Primary surface	Typical repository workflow
OpenCode (OpenCode 2026)	Terminal, desktop app, and IDE extension	Bring your own model provider (including local endpoints; see connecting OpenCode to local models) and edit the local checkout
Cline (Cline 2026)	VS Code extension, CLI, and SDK	Plan, then act with per-step approval; bring your own key, including Ollama
Kiro (Amazon Web Services 2026) (AWS)	IDE, CLI, web, and mobile	Spec-driven tasks locally, or delegate a web task that opens a pull request
Warp (Warp 2026)	Terminal app, standalone CLI, and cloud agents	Approve actions locally, or trigger cloud agents from Slack, Linear, or GitHub webhooks
OpenHands (OpenHands 2026)	Browser client, managed cloud, self-hosted backend, and CLI	Run headless or hosted; self-host when code must stay on your infrastructure
Devin (Cognition 2026) (Cognition)	Cloud agent and desktop editor	Delegate a task to an isolated cloud machine and review the pull request (see the harness landscape)
Ollama (Ollama 2026)	Local model runner (not an agent)	Serve open-weight models to any of the harnesses above (see Section 4)

Kepler is not an agent itself but an orchestration layer. It hosts agents you already use (Claude Code, Codex, Copilot, Cursor, OpenCode) rather than locking in one model, and builds on a decade of GitKraken plumbing for branches, worktrees, diffs, and merges. Where a single-agent platform handles one repo at a time, a Kepler Task can span many repos, and its Agent Graph visualizes every session, turn, tool call, and subagent live.

Where the agent runs

The platforms above sort into three execution models (measured 2026-09-09). The model decides where your code is copied, what the agent can reach, and how its work comes back to you.

- **Hosted sandbox.** The vendor provisions an isolated machine, clones the repository into it, and returns a branch or pull request. Jules clones the repository into a virtual machine

and submits a pull request after you approve its plan (Google 2026d); Cursor cloud agents clone from GitHub, GitLab, Azure DevOps, or Bitbucket, work on a separate branch, and push it back (Cursor 2026); Kiro web tasks and Warp cloud agents follow the same shape (Amazon Web Services 2026; Warp 2026), as do the Copilot coding agent, Codex cloud tasks, Devin, and OpenHands Cloud. Nothing runs on your machine, so the questions are what the sandbox can reach and whether the vendor's retention terms suit the repository.

- **Local checkout.** The agent runs on your workstation against the files already there, in a terminal (Claude Code, Codex CLI, OpenCode, Aider, Gemini CLI, Warp) or inside an editor (Cursor, Cline, Kiro, Copilot in VS Code). Code stays put, and the agent inherits whatever credentials and network access your shell has, which is why Section 19 asks for approval gates and a sandbox. Pairing one of these with Ollama keeps the model local too (Ollama 2026), at the cost of the hardware described in running agents offline.
- **Orchestration layer.** Kepler, Kiro Crew, and the Cline Kanban surface do not add a model of their own; they run several of the agents above in parallel worktrees or sessions and present the results for review. Reach for one only once a single agent is no longer the bottleneck (when orchestration helps).

Several vendors now ship all three, so the platform name alone no longer tells you where the code goes: Claude Code, Codex, Cursor, Kiro, and Warp each offer a local surface and a hosted one. Check the execution model of the specific surface you enable. The Windsurf editor is a naming trap of the same kind: `windsurf.com` now redirects to Devin Desktop, Cognition's rebranding of that editor (measured 2026-09-09; see the harness landscape).

For licensing, token-cost characteristics, and a decision table for choosing among these tools, see the harness landscape; for the open-weight and fully local end of the spectrum, see Section 4.

Before connecting any platform, check:

- whether code runs locally or in a vendor-managed environment;
- what repository, network, secret, and tool permissions it receives;
- whether changes arrive as a reviewable branch or pull request;
- which model providers and billing arrangements are supported; and
- whether your organization can retain the required audit trail.

Platform capabilities and commercial terms change quickly. Confirm current details in the linked official documentation before adopting one.

A separate category — named, persistent teammates with their own browser-and-shell computer, rather than a repository checkout — is reviewed in Grok Bot and Alternatives. Those products do not replace the platforms above.

4 Catalog and Comparison of Coding Agents

The coding agent ecosystem encompasses a spectrum of architectures ranging from proprietary cloud-managed assistants to fully autonomous, open-weight local harnesses (measured 2026-09-01; see Section 3 for platform overviews). Choosing an agent architecture requires evaluating autonomy, privacy, tool integration, and hardware constraints.

Autonomous Open-Weight Agents: Hermes Agent

Hermes Agent (developed by Nous Research) is an autonomous agent harness built around the Hermes series of open-weight models (such as Hermes 3). Unlike simple completion models, Hermes Agent specializes in structured function calling, multi-step tool execution, and reflective task planning. Key capabilities include:

- **Structured Tool Calling**: Native support for JSON schema function calling, enabling reliable invocation of compilers, test runners, and file system manipulators.
- **Multi-Turn Reasoning & Reflection**: Iterative planning loops that inspect tool execution outputs, handle runtime errors, and backtrack when encountering obstacles.
- **Deployment Flexibility**: Can execute against self-hosted open-weight model instances (via vLLM, Ollama, or TensorRT-LLM) as well as hosted API endpoints.

Local Coding Agents with Ollama

For air-gapped environments, strict data privacy requirements, or zero-marginal-cost development, developers pair local model runners like **Ollama** with dedicated agent harnesses (detailed setup and hardware sizing are covered in running agents offline):

- **Terminal Orchestrators**: Harnesses like **OpenCode** (see connecting OpenCode to local models), **Aider**, and **OpenHands** (formerly OpenDevin) connect directly to Ollama endpoints running open-weight coding models (such as `Qwen2.5-Coder`, `DeepSeek-Coder-V2`, or `Llama 3.3`), managing git commits, multi-file edits, and automated test-and-repair loops.
- **Editor Integrations**: Extensions such as **Continue** and **CodeCompanion** embed local Ollama models directly into VS Code, JetBrains IDEs, and Neovim, providing inline autocompletion and interactive chat without transmitting code to cloud APIs.

Comparative Taxonomy of Coding Agent Architectures

The following table compares the primary paradigms across the coding agent landscape:

Dimension	Cloud Frontier Agents (Antigravity, Claude Code)	Autonomous Open-Weight (Hermes Agent, OpenHands)	Local Editor Assistants (Continue + Ollama)
Primary Deployment	Cloud-hosted frontier API	Self-hosted or local server	Local workstation (Ollama or `llama.cpp`)
Autonomy Level	Semi-autonomous to fully autonomous	High multi-turn autonomy	Interactive / inline assistance
Privacy & Air-Gap	Cloud-dependent (enterprise VPC optional)	Fully air-gapped	100% offline & local
Tool Integration	MCP, terminal, browser, subagents	Bash execution, custom APIs	LSP, editor buffer manipulation
Compute Requirements	Minimal client hardware (network only)	High local GPU/VRAM (16GB-64GB+)	Moderate to high GPU/VRAM
Reasoning Capacity	Frontier reasoning & large context	Strong domain code reasoning	Bounded single-file reasoning

Trade-offs: Cloud Frontier vs. Local Open-Weight

When deciding between cloud frontier agents and local open-weight deployments:

1. **Complex Multi-File Refactoring**: Cloud frontier models (such as Claude Sonnet 4.5 or Gemini 3 Pro) currently maintain higher coherence across large, 50+ file refactors and architectural redesigns.
2. **Confidentiality and Compliance**: For proprietary codebases with strict regulatory constraints (HIPAA, defense, or sensitive enterprise IP), local agents powered by Ollama and Hermes Agent ensure that source code never leaves on-premises hardware (see running agents offline).
3. **Cost Predictability**: Local open-weight harnesses incur fixed hardware capital expense but zero marginal token costs, making them attractive for high-volume automated test-and-repair loops.

5 Where Current Practice Is Discussed: Reddit Communities

The practice of working with AI agents changes faster than any static page (these notes included) can track. Much of the current guidance circulates as forum discussion: tool comparisons, configuration recipes, failure reports, and launch announcements. This section catalogs

the Reddit communities where that discussion concentrates, grouped by what they are for, so you know where to look when a page here has gone stale. This section covers Reddit only: vendor documentation, project issue trackers, and the papers behind a technique are cited where the chapters use them, and the forum layer had no index here until this section.

> ⚠ **Counts and characterizations are snapshots**
>
> The member counts and the one-line characterizations in these tables are approximate and dated. They come from a directory compiled in August 2026 (issue #94) from third-party trackers (Prowlo, GummySearch, usefulai.com, and ReddTrends), which disagree with each other by tens of percent for the same community depending on crawl date. GummySearch stopped accepting new signups or renewals after 2025-11-30, so some cells it supplies reflect older snapshots. Reddit stopped showing member totals on community pages in late 2025, so a figure cannot be checked against the page itself. Read the counts as orders of magnitude, and when precision matters use a tracker that reads the official Reddit API (ReddTrends is one).

Dedicated to AI agents

Table 1: Communities dedicated to AI agents

Community	Members	What it is for
r/AI_Agents	~425k (Aug 2026)	The largest dedicated community: building and deploying LLM-based agents, framework comparisons, showcases, and launches. One marketing tracker describes it as roughly doubling over the preceding year.
r/aiagents	~118k	A separate community (no underscore) focused on tools, stack recommendations, and business automation.
r/AgentsOfAI	~98k	Showcase-friendly; known for its “I Made This” flair.
r/automation	~228k	Workflow automation with heavy agent crossover; not agent-only.

Frameworks and tools

Table 2: Framework and tool communities

Community	Members	What it is for
r/LLMDevs	~161k	Developer discussion of LLM applications, agent architectures, memory, and MCP builds.
r/mcp	~89k	The Model Context Protocol; a smaller r/modelcontextprotocol (~23k) covers the same ground.
r/LangChain	~87k	The largest framework community; much LangGraph discussion happens here too.
r/n8n	~220k	The n8n workflow platform, with heavy agent-workflow content.
r/AutoGPT	~21k	Grew out of the AutoGPT project into general agents and automation; tolerates launch posts.
r/AutoGenAI	~7k	Microsoft's AutoGen; small but active.
r/crewai	~1.6k	The CrewAI role-based framework.

The framework-specific communities are much smaller than the general agent ones, and r/PydanticAI, r/LangGraph, and r/LlamaIndex exist but had no reliable counts at compilation time. In practice, framework discussion concentrates in r/AI_Agents, r/LLMDevs, and r/LocalLLaMA rather than in the niche communities.

AI coding agents

Table 3: AI coding-agent communities

Community	Members	What it is for
r/ClaudeCode	~253k-387k	Claude Code specifically; trackers disagree widely on its size (usefulai.com listed ~253k in May 2026, while GummySearch listed ~387k).
r/ChatGPTCoding	~383k	The multi-tool coding community, where Claude Code, Cursor, Codex, and others are compared side by side.
r/vibecoding	~343k	Building applications by directing agents; heavy Claude Code, Cursor, and Bolt content.
r/VibeCodeDevs	~59k	Developers shipping real products with agents and automation.
r/Cursor	not verified	The Cursor editor and its agent features.

General AI communities with heavy agent discussion

Table 4: General AI communities

Community	Members	What it is for
r/ChatGPT	~11.6M	The largest applied-AI community; frequent agent and automation threads.
r/singularity	~4.0M	AI progress and futurism; skews toward hype.
r/MachineLearning	~3.0M	Research-grade, strictly moderated; agent papers.
r/OpenAI	~2.7M	OpenAI products, including its agent offerings.
r/ArtificialInteligence	~1.7-1.9M	Broad AI community (the misspelling is the actual handle).
r/artificial	~1.3M	Broad AI news and tools.

Community	Members	What it is for
r/ClaudeAI	~1.06M (Aug 2026)	Claude and Claude Code, with significant agentic crossover.
r/LocalLLaMA	~800k (Aug 2026)	The dominant local and open-weight model community; high signal on local agents and tooling.
r/LocalLLM	~207k	A second local-model community.

How to use this list

- **For the broadest agent discussion**, start with r/AI_Agents, then add r/AgentsOfAI and r/aiagents for showcases and use cases.
- **For framework work**, monitor r/AI_Agents, r/LLMDevs, and r/LocalLLaMA, where much of the framework discussion actually lands, and subscribe to a framework's own community once it reaches daily posting at a few tens of thousands of members.
- **For coding agents**, prioritize r/ClaudeCode and r/ChatGPTCoding, and add r/vibecoding for the builder angle.
- **For research and trend-spotting**, follow r/LocalLLaMA, r/MachineLearning, and r/singularity, filtering the last for hype.
- **Treat a community as a lead, not a source.** A forum thread is where a technique surfaces first; verify it against the tool's documentation before you rely on it, in the spirit of reviewing AI-generated work.

6 AI Agents and the Technological Singularity

The emergence of sophisticated AI agents has prompted discussions about whether we are witnessing or approaching a technological singularity. Understanding this concept helps contextualize the rapid evolution of AI tools and our responsibility in using them.

What is the technological singularity?

The technological singularity is a hypothetical future point when technological growth becomes uncontrollable and irreversible, resulting in unforeseeable changes to human civilization. The concept, popularized by mathematician Vernor Vinge and futurist Ray Kurzweil, typically involves the creation of artificial superintelligence that recursively improves itself, leading to an intelligence explosion beyond human comprehension or control.

Do current AI agents represent the singularity?

No, current AI coding agents (as of early 2026) do not represent the technological singularity.

While modern AI agents demonstrate impressive capabilities, they remain fundamentally different from the singularity scenario in several critical ways:

- **Limited autonomy**: Today's AI agents operate within strict boundaries and require human oversight. They cannot recursively improve their own core architecture or develop capabilities beyond their training.
- **Narrow intelligence**: AI coding agents are specialized tools designed for specific tasks. They lack general intelligence, self-awareness, or the ability to operate outside their designed domain.
- **Human dependency**: These agents require human developers to: review their work, provide direction, validate correctness, and make final decisions about their outputs.
- **No recursive self-improvement**: Current AI agents cannot fundamentally redesign themselves or create more advanced versions of themselves autonomously. Any improvements to AI systems still require human researchers and engineers. Anthropic's When AI builds itself says it is not there yet, while describing how much of its own AI development it already delegates to AI systems, and argues that recursive self-improvement could come sooner than institutions are prepared for.
- **Controlled development environment**: AI coding agents work in sandboxed environments with explicit permissions and constraints. They cannot independently acquire resources, modify their own constraints, or operate without human authorization.

Why this matters for responsible AI use

Understanding that current AI agents are powerful but limited tools—not autonomous superintelligences—has important implications:

- **Maintain appropriate skepticism**: AI agent outputs require the same critical review as any other tool-generated code.
- **Preserve human decision-making**: The responsibility for code quality, security, and correctness remains with human developers.
- **Continue skill development**: Using AI agents should enhance rather than replace human expertise.
- **Stay vigilant**: While current agents don't represent a singularity, the rapid pace of AI development requires ongoing attention to emerging capabilities and risks.

The value of AI coding agents lies in their ability to accelerate human productivity and learning, not in replacing human judgment or expertise. They are sophisticated tools that augment human capabilities while remaining under human control and oversight.

Further reading

For thoughtful perspectives on AI consciousness and intelligence, see Douglas Hofstadter's reflections in "I Thought I Was in an AI Apocalypse. Then I Started Looking Closer."

7 Relative Advantages of AI and Humans

AI coding agents and human coders have complementary strengths. Understanding these differences helps you decide when to delegate work to agents and when to handle tasks yourself.

Comparative Strengths: Humans vs. AI Agents

Table 5 summarizes the relative advantages of human coders and AI coding agents across different types of tasks:

Table 5: Relative advantages of humans and AI coding agents

Task Type	Humans	AI agents
Creative thinking	Humans excel at understanding context, handling ambiguous requirements, and thinking creatively about novel problems	AI agents struggle with ambiguous requirements and creative problem-solving in unfamiliar domains
Algorithmic thinking	Humans make mistakes when following repetitive instructions and may introduce inconsistencies	AI agents excel at executing well-defined, repetitive tasks with precision and consistency

---

Or, if you prefer a more visual representation:

Table 6: Relative advantages of humans and Agents

	Humans	AI Agents
Creative thinking		
Algorithmic thinking		

This pattern mirrors the evolution of programming itself. Just as almost no one writes machine code anymore because higher-level languages and compilers handle those details, most developers will increasingly spend less time writing low-level code. Instead, you'll describe what the system needs to do as clearly as possible, and AI agents will handle many of the

computational and coding details.

For most tasks, you won't need to step in and manipulate code yourself. However, you'll still need enough domain judgment to:

- Supervise and validate AI-generated code
- Handle edge cases that agents struggle with
- Make creative decisions about architecture and design
- Understand when agent suggestions are incorrect or suboptimal

Strong coding skills are one route to that judgment, and not the only one. A public field report from a non-programmer ("Vibe coded this game in four months", r/ClaudeCode, 2026-08-22; summarized in issue #98) describes four months of building a browser racing game with coding agents and no coding experience at all, crediting knowledge of game technology and design, plus long practice with language models, for the planning and the early catches that kept the project on track; by the author's own account, lacking that knowledge as well would have stalled the project after a couple of prompts. The requirement is the supervision, whichever background supplies it.

Future Developments: World Models

As AI technology advances, the distinction between these strengths may shift. Yann LeCun, 2019 Turing Award winner and AI researcher at Meta and NYU, advocates for developing "world models"—AI systems that understand and reason about the physical world, not just language patterns (LeCun 2022).

World models aim to give AI systems:

- **Persistent memory and reasoning**: Understanding that persists across interactions
- **Physical world understanding**: Reasoning about how things work in reality, not just in text
- **Better handling of ambiguity**: Using world knowledge to interpret unclear requirements

As these technologies mature, AI agents may become better at tasks requiring contextual understanding and creative problem-solving. This makes it even more important to develop strong supervision and validation skills now, so you can effectively work with increasingly capable AI systems.

8 Twelve Fixes for Claude Usage Limits

Issue #207 asked for a summary of a YouTube video, "Never Hit a Claude Limit Again — 12 Fixes Ranked by How Much Window They Buy", from the channel Hyperautomation Labs (Hyperautomation Labs 2026). The video is ten and a half minutes long and was published on 2026-08-16; this summary works from its auto-generated English transcript, fetched with `yt-dlp` on 2026-09-10, so wording attributed to the presenter below may carry speech-to-text errors. The first draft of the summary was produced by the Antigravity command-line program from that transcript and then checked against it by hand.

What the video claims

The presenter's claim is that a developer can work all day on the entry-level Claude plan without hitting the rolling five-hour usage wall, by managing where the tokens go rather than by buying a bigger plan. The video describes the limits as three layers that draw on one shared pool across the web app, the mobile app, Claude Code, and parallel sessions: a rolling five-hour window, an overall weekly cap, and a separate weekly cap for the flagship model tier. Anthropic does not publish the token arithmetic behind those limits, so everything in the video is inferred from the product's own usage displays.

The twelve fixes, in the video's order

The video counts down from the fix that buys the least window to the one that buys the most.

- **12. Read the meter.** Run the usage and context commands in Claude Code to watch the five-hour window, the weekly bars, and the per-category token split, so an active leak is visible while it is happening.
- **11. Kill the all-day session.** Clear the session between unrelated tasks; every prompt otherwise re-carries the whole conversation history.
- **10. Scope your asks.** Ask for the section that needs changing rather than a full regeneration; the presenter puts output tokens at about five times the cost of input tokens.
- **9. Reference file paths instead of pasting.** Point the agent at a file so it is read once on demand instead of sitting in the conversation permanently.
- **8. Compact on schedule.** Compact at task boundaries while the cache is warm, rather than resuming a stale session and paying for a cold, full-context compaction.
- **7. Shrink standing overhead.** Keep `CLAUDE.md` under about 200 lines and move workflows into skills. The presenter measured that toggling MCP servers off changed the startup cost by about 45 tokens, and concludes that the older advice to disable them no longer matters because tool definitions now load on demand.

- **6. Stop breaking the prompt cache.** Choose the model tier, effort level, and MCP connections at session start and change them only at task boundaries, so each request pays full price only for new tokens.
- **5. Use subagents as context firewalls.** Send noisy searches, file reads, and dead ends to subagents so only a short summary returns to the main session.
- **4. Configure cheaper models for subagents.** Put grunt-work subagents on a cheaper, faster tier. The transcript does not make the exact setting clear.
- **3. Route the main model deliberately.** Default to a workhorse tier and reserve the flagship tier for hard problems, because the flagship tier has its own weekly cap. The transcript is ambiguous about which model names map to which tier.
- **2. Schedule work around reset windows.** Batch heavy work into an active five-hour window and queue sessions rather than running four or more at once.
- **1. Install a prompt-discipline block.** Put standing instructions in `CLAUDE.md` that demand terse answers, minimal file reading, narrow tests, and no echoing of files back. The presenter offers the block itself as a download rather than reading it out, so its full text is not in the transcript.

The evidence, and its limits

The video's evidence is the presenter's own telemetry on one machine. Its headline measurement is that a fresh two-token prompt in Claude Code consumed more than 36,000 tokens of system prompt, tool definitions, and standing instructions before any work began. It also reports, from the local usage panel, that about 66 percent of one period's burn came from subagent sessions and about 18 percent from running four sessions in parallel. Those are single-user, single-day readings of a product whose accounting the vendor does not publish, taken from an auto-generated transcript; they are worth testing, not worth citing as fact.

Useful to us? Yes, and the lab has already paid for the lesson

The lab's own measurement agrees with the video's ranking. On 2026-09-09 a documentation sprint on this site fanned out roughly twenty Claude Code subagents at once and tripped the five-hour limit three times in one day, each time losing every worker mid-task; the fix that ended it was the video's number two, queuing sessions instead of running them in parallel, followed by routing work to a second vendor's quota through the Antigravity command-line program's headless `agy --print` lane. Three of the fixes map directly onto rules the lab already keeps in `Morrison-Lab/ai-config`:

- Fix 5 (subagents as context firewalls) is the standing "use subagents when helpful" rule, and fix 4 (cheaper subagent models) is its model-routing section.
- Fix 7 (short `CLAUDE.md`, workflows in skills) is what the corpus's skills directory is for, and the corpus has an instrument, `check-context-closure.py`, that measures the always-loaded pool the presenter is warning about.

- Fix 2 (queue rather than parallelize) is what the sprint above settled on: after the third exhaustion the concurrency was capped at two Claude subagents at once, and the remaining work was routed to a second vendor's quota.

Two of the fixes are not worth adopting as stated. The prompt-discipline block (fix 1) is a promotional download whose text the video withholds, and "terse answers, minimal file reading" conflicts with the lab's review rules, which pay tokens for verification on purpose. And the MCP finding (fix 7) is one measurement on one build; re-measure it on your own setup before removing servers you rely on.

9 Delegating Bulk Judgments with Quicksilver

Issue #247 notes Quicksilver, an open-source Claude Code skill and plugin authored by Udit Akhouri (Akhouri 2026). Quicksilver addresses a central inefficiency in agent workflows: paying an expensive frontier model to skim hundreds of files, logs, or tickets just to determine which subset actually requires deep analysis.

Delegating bulk judgment to lightweight models

When asked questions such as "which files handle authentication?" or "which lines report errors?", a frontier model typically reads every candidate file or log line into its context window. This process burns thousands of tokens on routine filtering, fills the context with irrelevant noise, and rapidly approaches session usage limits.

Quicksilver restructures this workflow by routing candidate items to Jev, a lightweight model from TypeSafe designed for high-throughput System One judgment. Jev evaluates items in parallel and returns typed verdicts (boolean flags, categorical labels, or scores) in approximately one second at a fraction of frontier model pricing ($0.042 per million input tokens, with free output tokens). Claude receives only a filtered shortlist of relevant items with confidence ratings, reserving its reasoning tokens for synthesis, refactoring, and code changes.

Measured benchmark results

On a benchmark of twelve real-world tasks across public datasets (including supercomputer logs, `Banking77` intent classification, UCI SMS spam, `SST-2` sentiment analysis, the Hono repository, and `lodash` source code) (Akhouri 2026):

- **Token reduction:** Claude token consumption was reduced by an average of 86 percent (median 82 percent).
- **Execution speed:** Tasks involving needle-in-haystack search and log triage completed up to 20 times faster.

- **Accuracy parity:** Quicksilver matched Claude's baseline accuracy on 8 of the 12 evaluation tasks.

Appropriate use cases and boundary conditions

Quicksilver is most effective when tasks can be formulated as a narrow, typed judgment:

- **Needle-in-haystack scans:** Locating relevant implementation files across a multi-hundred-file repository, pinpointing rare error signatures in multi-thousand-line server logs, or isolating target helper routines in large single-file modules.
- **Structured ticket and log classification:** Categorizing user requests or triage items against closed label sets.
- **Borderline case review:** Quicksilver flags ambiguous or borderline items (annotated with a question mark), enabling Claude to inspect only the uncertain cases rather than the entire corpus.

Quicksilver is not intended for authoring code, complex multi-step reasoning, or tasks that deterministic tools like `grep` can answer directly.

10 Andrew Ng on Learning to Code in the Age of AI

Issue #244 asked for a summary of a YouTube video, "Andrew Ng: One Skill to Stay Relevant in the Age of AI", from the channel Stanford Online (Stanford Online and Ng 2026). The video is an eighty-nine-second interview published on 2026-09-22, in which Andrew Ng (Adjunct Professor at Stanford University and founder of DeepLearning.AI) responds to the question: "What's one skill professionals should start learning now if they want to stay relevant in the age of AI?"

The core argument: everyone should learn to code

Ng opens with what he acknowledges is a controversial stance: everyone should learn to code. His thesis is that a noticeable productivity gap is already widening between professionals who can build custom software and those who cannot, across all organizational roles rather than only within software engineering.

Directing computers vs. writing syntax by hand

Crucially, Ng clarifies that learning to code does not mean typing out every line of syntax manually. He notes that he rarely writes code entirely by hand himself anymore. Instead, the enduring core skill is knowing how to instruct a computer with precision, and code remains the native language through which computers are directed.

In Ng's view, AI-assisted coding transforms programming into a universal capability:

- **Marketing without engineering bottlenecks:** Top marketers no longer wait in an engineering backlog for a dedicated developer to build or update a website; they build and deploy what they need directly using AI assistance.
- **Recruiting with custom automation:** Top recruiters write automated scripts to screen and parse resumes against specific role requirements instead of manually reviewing hundreds of candidates by hand.

Countering the advice to abandon coding

A prevalent piece of executive and business advice suggests that learning to code is a waste of time because AI will automate software development away. Ng argues that this perspective has the causality backwards. Rather than making programming obsolete, AI assistance has lowered the barrier to entry enough that coding is now practical for everyone to learn. The ability to specify logic, structure data, and guide an AI coding agent to produce tailored tools amplifies individual effectiveness in every profession.

11 When to use a coding agent

Coding agent sessions are currently[1] considered "premium requests", which are limited resources; see https://github.com/features/copilot/plans for details. So, use coding agents sparingly. Use them for complex changes that would be difficult or time-consuming for you to complete by hand. Coding agents also take time to get configured for work, every time you make a request. See https://docs.github.com/en/copilot/how-tos/use-copilot-agents/coding-agent/customize-the-agent-environment#preinstalling-tools-or-dependencies-in-copilots-environment for ways to reduce that startup time, but it will never be 0. If you can complete the task faster than the coding agent can, you should probably do it yourself. For example, when you have errors in the spell-check or lint workflows, you can often fix them faster than Copilot can. Similarly, when reviewing Copilot's PRs, you can often make direct changes to the branch faster than you could write clear review comments and get Copilot to address them.

Also, the less we practice, the weaker our skills get, and the harder it is for us to supervise the agents and make sure they are actually doing what we want them to do, the way we want them to do it. You should exercise your own coding skills regularly, just like you would for any other skill you want to maintain.

[1] 2026-01-10

12 Deep Research Modes

Every major assistant now ships a "deep research" mode: a long-running agent that plans a search strategy, runs dozens of queries, reads the results, and writes a cited report (measured 2026-09-09). These modes sit between a single web search and a coding agent. They do not edit files or run your code, but they read far more sources per question than a chat turn does, and they hand back something closer to a literature memo than an answer.

This section compares five of them from their vendors' own documentation:

- Claude Research (Anthropic)
- Gemini Deep Research (Google)
- ChatGPT deep research (OpenAI)
- Researcher in Microsoft 365 Copilot (Microsoft)
- Perplexity Research (Perplexity)

For each we record what it does, which plans include it and with what limits, how long a run takes, what the output looks like, which connected sources it can read, and whether the same capability is reachable from an API. All figures are as published on 2026-09-09; vendors change plan limits often, so re-check before relying on one.

Claude Research

Research (Anthropic 2026b) is available on the paid Claude plans (Pro, Max, Team, and Enterprise) in the web, desktop, and mobile apps. Web search must be switched on for Research to work. Claude runs a chain of searches that build on each other and returns an answer with inline citations. Anthropic's guidance on choosing between web search, extended thinking, and Research (Anthropic 2026d) places Research at "five or more tool calls over 1-3 minutes", with web search for one- or two-query lookups and extended thinking for reasoning that needs no new information.

Research draws on the same connectors as the rest of the app: when Gmail, Google Calendar, and Google Docs are connected, it searches those alongside the web (Anthropic 2026b). There is no separate Research quota; runs count against the plan's ordinary conversation limits, and the help page warns that a run "may consume limits faster" because it retrieves many sources.

The Claude API has no Research endpoint. What it exposes is the web search tool (Anthropic 2026c): a server-side tool that searches, returns results with citations, and can be capped with `max_uses` or restricted to `allowed_domains`. It is billed at $10 per 1,000 searches plus token costs. A multi-search research loop over that tool is something you build, not something you call.

Gemini Deep Research

Deep Research in the Gemini app (Google 2026e) searches Google by default, and you can add Gmail, Drive, uploaded files, and NotebookLM notebooks as sources. A report "usually takes about 5-10 minutes to generate", longer for complex topics. The report can be exported to Google Docs or turned into an Audio Overview. All users can run reports; Google AI Pro and Ultra subscribers can generate them with the Pro model, and Ultra reports may include charts, diagrams, and interactive simulators.

Limits are compute-based rather than a per-report count (Google 2026b): the allowance refreshes every 5 hours up to a weekly cap, with AI Plus at 2x the free allowance, AI Pro at 4x, and AI Ultra at 5x or 20x the Pro allowance depending on the subscription. Deep Research is listed as available on every tier, but the same page notes that for accounts without a paid plan compute-heavy features like Deep Research "may be unavailable during periods of high demand".

Google is the one vendor here that sells the app feature and the API feature under the same name. The Gemini Deep Research agent (Google 2026c) runs only through the Interactions API, must be launched with `background=true`, and ships as two agents: `deep-research-preview-04-2026` and `deep-research-max-preview-04-2026`. The Max variant, built on Gemini 3.1 Pro, is positioned for exhaustive batch work such as a nightly job that writes due-diligence reports (Google 2026a). By default the agent gets Google Search, URL context, and code execution; it accepts documents as input and can call remote MCP servers. A run is capped at 60 minutes, and the docs estimate $1-3 per standard task and $3-7 per Max task.

ChatGPT deep research

Deep research in ChatGPT (OpenAI 2026b) is the one mode here that puts a plan in front of you before it starts. ChatGPT may ask clarifying questions, drafts a research plan you can edit, lets you watch and interrupt the run to redirect it, and finishes with a full-screen report carrying a table of contents, a sources section, and an activity history. Reports download as Markdown, Word, or PDF.

Sources are the public web, files you upload, and connected apps such as Google Drive or SharePoint where your plan and workspace allow them; deep research uses only an app's read actions. You can also restrict a run to named sites, or prioritize them while still allowing full-web search. Usage "varies by plan" and shows in an in-product counter; plans with a fixed monthly allowance reset 30 days from first use, and availability depends on country. Enterprise and Edu admins gate access by role.

The API version (OpenAI 2026a) runs through the Responses API with model `o3-deep-research` or `o4-mini-deep-research`. OpenAI recommends background mode because runs "can take tens of minutes", and background mode is incompatible with zero-data-retention agreements. Available tools are web search, file search over at most two vector stores, remote

MCP servers exposing a search/fetch interface, and the code interpreter. Output carries inline citations as annotations with a URL, title, and character offsets.

Microsoft 365 Copilot Researcher and Think Deeper

Microsoft has consolidated its offerings into one agent. The consumer "Deep Research" mode was retired from the Copilot app on 2026-08-18, and Microsoft 365 Premium subscribers are directed to Researcher (Microsoft 2026c, 2026b) instead. Researcher is also included for business and enterprise tenants with a Copilot add-on license, and admins can block it in the Microsoft 365 admin center.

Researcher's distinguishing source is your work graph: it reads files, emails, meetings, and Teams chats you already have access to, plus Microsoft Graph connectors, and uses the Bing index for the web (Microsoft 2026d). Web search can be disabled tenant-wide, but there is no allow-list or block-list of sites below that. It asks clarifying questions before running, and the FAQ gives run times of "under five minutes for simple queries" and "10 to 45 minutes for highly complex ones". Output is a report with headings, visuals, and cited sources; PowerPoint and PDF export are listed as "to be released soon". The hard limit is 25 queries per user per month. Tenants can enable Anthropic's Claude models inside Researcher, and the model picker is available when Researcher is opened as its own agent. The FAQ states that Researcher cannot be customized or extended through Copilot Studio, and none of the pages we read describe a programmatic route to it (Microsoft 2026e).

Think Deeper is not a research mode. It is one of the conversation modes (Microsoft 2026a) in the consumer Copilot app: a reasoning pass that "takes up to 10 seconds" and does no extra retrieval, available to all users with priority for Microsoft 365 subscribers when capacity is short. Treat it as the counterpart of Claude's extended thinking, not of Researcher.

Perplexity Research

Research mode (Perplexity 2026b) runs "dozens of searches, reads hundreds of sources", completes most tasks in under 3 minutes, and takes around 4 to 5 minutes to deliver the report. Reports export to PDF or a document, or become a shareable Perplexity Page. Free accounts get limited access and Pro subscribers get extended access; the help page gives no counts. You cannot choose the model in Research mode. The Advanced Deep Research (Perplexity 2026c) update added a code sandbox, uploaded-document input, clarifying questions, follow-up questions while a run is in progress, and reports that stream into an editable file. It also names the underlying model by plan. Which model each plan gets has changed more than once, and Perplexity's help-center pages refuse automated requests, so this section does not reproduce the mapping: check the linked page before relying on a particular model being behind your plan.

The API equivalent is the `sonar-deep-research` model (Perplexity 2026a), priced at $2 per million input tokens, $8 per million output tokens, $2 per million citation tokens, $3 per million

reasoning tokens, and $5 per 1,000 searches, with a 128K context window. The docs carry a deprecation notice moving the Sonar chat-completions interface to the Agent API, with the old interface supported until 2026-09-27.

Comparison

	Claude Research	Gemini Deep Research	ChatGPT deep research	Copilot Researcher	Perplexity Research
Plans	Pro, Max, Team, Enterprise	All tiers; Pro model on AI Pro and Ultra	Varies by plan and country	M365 Premium; Copilot add-on licenses	Free (limited), Pro (extended)
Limit	Shared conversation limits	Compute-based, 5-hour refresh, weekly cap	In-product counter; monthly allowance on some plans	25 queries per user per month	Not published
Run time	1-3 minutes	5-10 minutes	Not published for the app; "tens of minutes" via API	Under 5 minutes to 45 minutes	3-5 minutes
Plan review	No	No	Editable plan; run can be interrupted	Clarifying questions	Clarifying questions
Output	Answer with inline citations	Report; Google Docs export; Audio Overview	Report with table of contents and sources; Markdown, Word, PDF	Report with visuals and citations	Report; PDF, document, Perplexity Page
Connected sources	Gmail, Calendar, Docs connectors	Gmail, Drive, uploads, NotebookLM	Uploads, Drive, SharePoint, other apps; site restriction	Files, email, meetings, chats, Graph connectors	Uploaded documents

	Claude Research	Gemini Deep Research	ChatGPT deep research	Copilot Researcher	Perplexity Research
API	Web search tool only ($10 per 1,000 searches)	Interactions API; two agents; 60-minute cap; $1-7 per task	Responses API; `o3-deep-research`, `o4-mini-deep-research`	None documented	`sonar-deep-research`

Useful to us?

Yes, for a specific slice of work, and no as a substitute for the agents in Section 4.

Reach for a research mode when the question is about the world rather than about your repository: a literature scan before a grant section, a comparison of vendors or packages you have not yet chosen between, or a check on what a regulator or funding agency currently requires. The modes read many more pages than a coding agent will in one turn, they hand back citations you can audit, and several export straight into the document formats we already draft in (see collaborative workspaces for the workspace side). Researcher is the odd one out: its value is reading your own institution's email and files, which none of the others can see, and that is also why its output stays inside the Microsoft 365 boundary.

Reach for a coding agent with web search instead when the answer must touch the code: a dependency upgrade that needs the changelog read *and* applied, a CI failure whose fix lives in a vendor's docs, or anything where the deliverable is a diff rather than a memo. Section 11 covers that decision in general. A coding agent can also chain research and action in one run, whereas every research mode above stops at the report.

Two cautions carry across all five:

- **The limits are the product.** Monthly caps of 25 (Copilot) or compute pools that refresh every 5 hours (Gemini) mean a research mode is not a tool to leave running in a loop. For batch or scheduled research, use the API routes, and budget them like any other agent spend (spend management).
- **A cited report is not a verified one.** Every mode cites, and none checks that the cited page supports the sentence. Spot-check the citations you intend to reuse before they reach a manuscript, exactly as the fact-check rules in this manual require of any AI-drafted prose.

13 How to Work with Coding Agents

Coding agents can be accessed through several interfaces, each with different trade-offs for task size, feedback speed, and collaboration style.

Ways to interact with coding agents

The main differences are where the agent runs, how much repository context it can access, and how directly it can apply changes.

- **Cloud coding agents (GitHub Issues/PR workflow)**: Best for larger, asynchronous tasks (for example, multi-file refactoring, dependency updates, or documentation changes spanning several chapters). The agent runs in a managed cloud environment, works through an issue, and proposes changes in a pull request for review.
- **CLI agents (terminal-first workflows)**: Best for rapid, local iteration. You stay close to shell tools and local files, with faster back-and-forth for debugging, refactoring, and test-driven edits.
- **Chat/app agents (for example Claude or Codex-style chat interfaces)**: Best for planning, design discussion, and drafting code ideas. They can be very strong thought partners, but often need more manual copy/edit/execute steps unless tightly integrated with your repository tools.
- **IDE-integrated agents**: Best for mixed human and agent editing. They combine conversational help with direct file edits, code navigation, and in-editor testing loops.

In practice, teams often combine these modes: use app or IDE chat to refine requirements, then hand implementation to a cloud or CLI agent, and finish with human review.

The following sections focus on GitHub Copilot's specific workflow for assigning and managing coding tasks.

Assigning Issues to Copilot

You can assign GitHub Issues directly to `@copilot` just like you would assign to a human collaborator:

1. **On GitHub.com**: Navigate to an issue and assign it to Copilot in the assignees section
2. **In VS Code**: In the GitHub Pull Requests or Issues view, right-click an issue and select "Assign to Copilot"
3. **From Copilot Chat**: Delegate tasks to Copilot directly from the chat interface in supported editors

The Agent Workflow

Once assigned an issue, the coding agent follows an autonomous workflow:

1. **Analysis**: Reviews the issue description, related discussions, repository instructions, and codebase context
2. **Planning**: Determines what changes are needed and creates a work plan
3. **Development**: Works in an isolated GitHub Actions environment, modifies code, runs tests and linters, and validates changes
4. **Pull Request Creation**: Creates a draft pull request with implemented changes, audit logs, and a summary of modifications
5. **Review and Iteration**: You review the PR and can request changes; the agent will iterate based on your feedback

Collaborating with Coding Agents

Between iterations of asking coding agents to extend a PR, human collaborators can also push changes directly to the PR branch. This allows for a collaborative workflow where both humans and agents contribute:

- **Human contributions**: You can make quick fixes, add content, or refine the agent's work by pushing commits to the same branch
- **Agent iterations**: After your changes, you can ask the agent to continue working on additional requirements

Important: Try to avoid pushing changes while the coding agent is actively working. Simultaneous edits can produce conflicting diffs that:

- Need to be manually resolved
- May confuse both human and AI collaborators
- Could result in lost work or merge conflicts

Best practice: Wait for the agent to complete its current iteration (indicated by the PR being updated) before pushing your own changes to the branch. Then assign new work to the agent for the next iteration.

Directly Prompting for Pull Requests

You can also prompt Copilot to create pull requests without first creating an issue:

- Use Copilot Chat in your editor to describe the changes you want
- The agent will analyze your request and create a pull request
- This is useful for quick fixes or well-defined tasks

Important Safeguards

- **Human approval required**: Coding agents cannot merge their own changes
- **Branch restrictions**: Agents can only push to their own branches (e.g., `copilot/*`)
- **Full transparency**: All agent actions are logged and visible in the PR

Workflow Approval Requirements

When GitHub Copilot creates or updates a pull request, it cannot automatically trigger GitHub Actions workflows. **You must manually approve each workflow run** by clicking the approval button in the Actions tab or on the PR.

This manual approval requirement is a security measure that prevents potentially malicious or unintended code execution. Because Copilot can modify any file in the repository—including workflow files themselves or scripts called by workflows—allowing automatic workflow execution could create security vulnerabilities.

Key points:

- **No automatic approval**: There is currently no way to bypass manual workflow approval for Copilot PRs, even if you are the repository owner
- **Security reasoning**: Copilot could modify workflow files (`.github/workflows/*.yml`) or scripts they execute, potentially injecting malicious code
- **Impact on workflow**: This means you need to actively monitor and approve workflow runs as Copilot iterates on your issue, which can slow down the development cycle

Workaround considerations:

Some users have discussed using Personal Access Tokens (PATs) to allow Copilot to trigger workflows on your behalf, but this approach has security implications and should be carefully evaluated before implementation.

For more details and community discussion about this limitation, see:

- GitHub Community Discussion #162826: Discussion about workflow approval requirements
- GitHub Community Discussion #183966: Product feedback on this topic

For detailed instructions, see GitHub Copilot coding agent documentation.

14 Useful Prompt Formats

When working with coding agents, using clear and specific prompts helps achieve better results. Here are some useful prompt formats that you can use when requesting assistance from coding agents:

Common Task Patterns

Tidying up code:

- "tidy up [file, function, module, whole project]"
- Useful for improving code organization, consistency, and readability
- Example: "tidy up the data processing module"

Addressing failing workflows:

- "address failing workflows"
- Helps fix continuous integration (CI) failures, build errors, or test failures
- Example: "address failing workflows in the GitHub Actions pipeline"

Decomposing code:

- "decompose [function, quarto-file, etc]"
- Breaks down large or complex code into smaller, more manageable pieces
- Example: "decompose this analysis function into separate helper functions"

Updating content:

- "update [links, content, etc]"
- Refreshes outdated information, fixes broken links, or modernizes code
- Example: "update all package URLs in the documentation"

Expanding documentation:

- "expand [a section in a document]"
- Adds more detail, examples, or explanation to existing content
- Example: "expand the section on data validation with practical examples"

Condensing content:

- "condense [a section in a document]"
- Reduces verbosity while preserving essential information
- Example: "condense the installation instructions to be more concise"

Clarifying content:

- "clarify [a section in a document]"
- Improves clarity, removes ambiguity, or simplifies complex explanations
- Example: "clarify the explanation of the analysis workflow"

Tips for Effective Prompts

- **Be specific**: Include file names, function names, or specific sections when possible
- **Provide context**: Explain what you want to achieve and why
- **Set boundaries**: Specify what should or shouldn't change
- **Request validation**: Ask the agent to test or verify its changes when appropriate

15 Editing with `.docx` files

GitHub Copilot coding agents can read Microsoft Word (`.docx`) files, including tracked changes and comments. This enables a hybrid editing workflow where:

1. Lab members can export Quarto content to Word format for review
2. Reviewers can make edits, add tracked changes, and insert comments in Word
3. Coding agents can read the `.docx` file and translate the edits back to Quarto format

When using this workflow, make sure to explicitly instruct the coding agent to:

- Examine and apply all tracked changes in the `.docx` file
- Read and address all comments in the `.docx` file
- Translate edits from Word formatting to appropriate Quarto/markdown syntax

This approach makes it easier for collaborators who are more comfortable with Word to contribute while maintaining the source files in Quarto format.

Known Issue: "Document 1" Warning in Word

When opening DOCX files generated by Quarto (including this site), Microsoft Word may display a warning message and open the file with the title "Document 1" instead of the actual filename. Word may also require you to save the file before you can add comments or track changes.

This is a known limitation with how Quarto generates DOCX files. The issue is being tracked in the Quarto project:

- Quarto CLI Issue #6357
- Quarto Discussion #6544
- Quarto CLI Issue #10587

Workaround: If you are the author generating the DOCX file from Quarto, follow these steps before sharing with collaborators:

1. Open the generated DOCX file in Microsoft Word
2. Immediately save the file (File → Save, or Ctrl+S/Cmd+S)

3. Close and re-open the file to verify it no longer shows "Document 1"
4. Share this saved version with collaborators

This one-time step ensures that when collaborators open the file, they won't see the "Document 1" warning and can immediately add comments and track changes without issues.

16 Copilot Instructions for this Repository

A `.github/copilot-instructions.md` file contains repository-specific instructions and guidelines for GitHub Copilot coding agents. This file helps ensure that AI-generated contributions follow the project's formatting standards, coding conventions, and documentation practices.

For a Quarto-based repository like this one, the copilot instructions file typically specifies:

- Markdown and Quarto formatting rules (e.g., blank lines before lists, line breaks in prose)
- R code style guidelines (e.g., using native pipe `|>`, following tidyverse style)
- File organization patterns (e.g., using Quarto includes for modular content)
- How to work with DOCX files for hybrid editing workflows
- Version control best practices (e.g., do not commit rendered artifacts that are generated only for preview/review)
- Repository-specific best practices

By having these instructions in `.github/copilot-instructions.md`, you ensure that coding agents produce consistent, high-quality contributions that align with the project's established practices. This reduces the review burden and helps maintain consistency across all contributions, whether made by humans or AI assistants.

See this repository's own `.github/copilot-instructions.md` for a working example.

17 Addressing Failing GitHub Actions Workflows

When GitHub Actions workflows fail, you can use Copilot to help diagnose and fix the issues. However, it's important to use the right prompts depending on whether the problem is in your code or in the workflow configuration itself.

Scenario 1: Code Issues Found by Workflows (Most Common)

When to use: The workflow is functioning correctly, but it's detecting problems in your code (e.g., failing tests, linting errors, build failures).

What you want: Fix the code issues without modifying the workflow files themselves.

Recommended prompts:

- "fix the code issues found by the failing workflows"
- "address the linting errors reported in the GitHub Actions checks"
- "fix the test failures in the CI pipeline"
- "resolve the build errors shown in the workflow logs"

Example: If your R package has failing tests detected by `usethis::use_github_action("check-standard")`, you want Copilot to fix the test failures in your R code, not modify the workflow YAML file.

Why this matters: These prompts make it clear that you want code changes, not workflow changes. This helps prevent the agent from unnecessarily modifying your carefully-configured CI/CD pipeline.

Scenario 2: Issues with Workflow Files Themselves

When to use: The workflow configuration itself has problems (e.g., syntax errors in YAML, incorrect job definitions, outdated actions).

What you want: Fix the workflow files, but with extreme caution due to security implications.

Recommended prompts:

- "fix the syntax error in the GitHub Actions workflow file at line X"
- "update the workflow to use the latest version of action Y"
- "correct the job configuration in `.github/workflows/check-standard.yaml`"

Important considerations:

> ⚠ Warning
>
> **Security Warning**
> Workflow files have access to repository secrets and can execute arbitrary code. Before accepting any changes to workflow files:
>
> 1. **Review every line** of the proposed changes
> 2. **Verify** the changes only address the specific issue
> 3. **Check** that no new secret access or command execution has been added
> 4. **Test** in a safe environment if possible
>
> See Section 19 for more details on workflow file security.

When to do it yourself: Workflow syntax errors and configuration issues are often faster to fix manually than with Copilot, especially if you're familiar with GitHub Actions. See Section 11 for more guidance.

Scenario 3: Uncertain Which Scenario Applies

When to use: You're not sure whether the failure is due to code issues or workflow configuration problems.

Recommended approach:

1. **First, examine the workflow logs**:
   - Look at the error messages in the GitHub Actions tab
   - Identify whether the error is in your code or the workflow itself
   - Common code issues: test failures, linting errors, compilation errors
   - Common workflow issues: YAML syntax errors, missing actions, permission errors
2. **Use a diagnostic prompt**:
   - "examine the failing workflow logs and identify whether the issue is in the code or the workflow configuration"
   - "diagnose the root cause of the workflow failure"
3. **Then use the appropriate scenario above**: Once you understand the issue, use the specific prompts from Scenario 1 or 2.

Example workflow:

```
1. Prompt: "examine the failing workflow logs and identify the issue"
2. Copilot responds: "The workflow is failing because of linting errors
   in src/analysis.R"
3. Prompt: "fix the linting errors in src/analysis.R"
```

Additional Resources

- See the UCD-SERG Lab Manual's continuous integration chapter for setting up GitHub Actions workflows
- See Section 19 and Section 18 for security considerations with workflow files
- See Section 11 for guidance on when to use Copilot vs. fixing issues yourself
- See the GitHub Actions documentation for workflow syntax and troubleshooting

18 Benefits and Hazards

Coding agents are powerful programs that can work autonomously. They create pull requests that propose changes to the code in our repositories, potentially including their own configuration files and our automated workflows. They can work powerfully on our behalf, but they

require careful oversight and control to ensure they serve our interests and that we understand the consequences of their actions.

Coding agents offer several advantages:

- **Built-in transparency**: Coding agents create a clear record of their role in your work through commit history and code suggestions
- **Context-aware suggestions**: Coding agents understand your codebase and can make contextually relevant suggestions
- **Integration with version control**: Using coding agents within GitHub ensures that AI-assisted changes are tracked alongside all other code changes
- **Interactive workflow**: Coding agents' interactive nature encourages you to review and modify suggestions rather than blindly accepting them
- **Accelerated development**: Coding agents can help you write boilerplate code, refactor existing code, and implement common patterns more quickly
- **Learning opportunities**: Coding agents can suggest approaches or techniques you may not have considered, helping you expand your coding knowledge

However, coding agents also come with significant hazards:

- **Over-reliance**: Depending too heavily on coding agents can atrophy your coding skills and understanding
- **Subtle bugs**: AI-generated code may contain logic errors that are not immediately obvious
- **Security vulnerabilities**: Coding agents may introduce insecure patterns or fail to follow security best practices
- **Inappropriate solutions**: AI may suggest solutions that work but are not optimal for your specific research context or constraints
- **Hidden biases**: Coding agents may perpetuate coding patterns or approaches that reflect biases in their training data
- **False confidence**: Well-formatted, professional-looking code from AI can mask underlying problems and reduce critical review
- **Workflow manipulation risks**: Coding agents that modify CI/CD workflows (`.github/workflows/*.yml`) or setup configurations can inadvertently or maliciously compromise repository security, expose secrets, or execute harmful commands

Further reading/viewing

- *I Robot* (Asimov 1950)
- *Dune* (Herbert 1965)
- *2001* (1968)
- *Terminator 3* (2003)
- *The Matrix* (1999)
- *Blade Runner* (1982)
- *WarGames* (1983)
- *Battlestar Galactica* (2004) (*Battlestar Galactica* 2004)
- *Ender's Game* (Card 1985)
- "The Humans are Dead" (Flight of the Conchords 2007)



Figure 1: Agents

19 Best Practices for Safe and Successful Use

To work with coding agents safely and successfully:

1. **Maintain active supervision**: Never assume AI-generated code is correct. Review every line critically.
2. **Understand before accepting**: If you don't understand what the code does, don't use it. Take time to learn or ask a colleague.
3. **Test thoroughly**: AI-generated code must be tested as rigorously as code you write yourself. Don't skip testing because "the AI wrote it." Budget for it, too. Two commenters on the same field report ("Vibe coded this game in four months", r/ClaudeCode,

2026-08-22; summarized in issue #98) independently named manual testing as the dominant cost once generation is cheap. One put it as needing either deep domain experience or heavy testing to get a defect-free result.

4. **Start small**: Begin with small, well-defined tasks to build confidence and understanding of the agent's capabilities and limitations.
5. **Verify logic and assumptions**: Check that the AI hasn't made incorrect assumptions about your data, requirements, or scientific context.
6. **Review for security**: Explicitly check for security issues, especially when handling sensitive data or user input.
7. **Iterate and refine**: Use coding agents as a starting point, not an endpoint. Refine and improve the generated code.
8. **Maintain coding practice**: Regularly write code yourself to maintain and develop your skills. Don't let the agent do everything.

> **UC Davis campus guidance**
>
> UC Davis Student Affairs also provides guidance on the Responsible Use of Artificial Intelligence (AI). That page provides UC Davis-specific guidance on AI tool selection, campus training, and careful handling of sensitive data. Their recommendations, like ours, include careful validation, active supervision, and protection of confidential information.

> **Critical: Exercise Extreme Caution with Workflow Files**
>
> Be especially careful when allowing coding agents to edit GitHub Actions workflows or CI/CD configurations. These files control automated processes that can:
>
> - Access secrets and credentials
> - Deploy code to production
> - Execute arbitrary commands in your repository
>
> **Never** allow a coding agent to edit workflow files (especially `.github/workflows/*.yml` or `copilot-setup-steps.yml`) without thorough manual review. Before approving any workflow run, always check if the workflow files themselves have been modified. Malicious or erroneous changes to workflows can compromise your entire repository and its secrets.

When using coding agents, work interactively with the AI suggestions: review, modify, and test them rather than accepting them wholesale. This interactive approach helps ensure code quality and deepens your understanding of the code.

Remember: AI tools are assistants, not replacements for your expertise and judgment. The quality and correctness of your work remains your responsibility.

20 Citadel and Levels of Claude Code Use

Every lab that uses a coding agent for more than one-off edits eventually builds something around it: instruction files, then skills, then hooks, then scripts that watch pull requests. Customizing an Agent maps those mechanisms one at a time. This section looks at the question from the other end: what does a *complete* layer look like when someone builds one deliberately, and how much of it does the lab actually need?

The example is Citadel (Gammon 2026d), an open-source "operating layer" for Claude Code and OpenAI Codex. Issue #102 paired it with a r/ClaudeAI post describing five maturity levels of Claude Code use. That post could not be fetched from the network this section was written on (every route to `reddit.com` was blocked, measured 2026-09-09), so the five-level model is tracked separately in #227, and the ladder discussed here is Citadel's own.

What Citadel is

Citadel is a plugin, not a harness (harnesses). It installs into an existing Claude Code or Codex session through the runtime's plugin marketplace, pinned to a release tag, and then adds four things around the agent you already run (Gammon 2026d):

- a single natural-language entry point, `/do`, that routes a request to a skill,
- repository-local state under `.planning/` that survives the end of a session,
- approval boundaries and verification hooks around multi-step changes, and
- coordination for several agents working in parallel worktrees.

It is MIT-licensed JavaScript requiring Node.js 22 or later, and it is essentially one person's project: of roughly 630 commits, 600 are by the author, 19 by Dependabot, and the rest by three occasional contributors (measured 2026-09-09). The repository was created in March 2026, has 920 stars and 82 forks, and its latest release, `v1.3.5`, dates from 2026-08-13. The release ships 48 skills, 35 hook scripts across 29 lifecycle events, and 7 subagent definitions.

The README frames the operating loop as five public states: **Request**, **Run**, **Evidence**, **Needs You**, and **Resume**. The last two are the interesting ones. "Needs You" is a deliberate stop that names the exact approval, conflict, or missing evidence required before the agent continues. "Resume" means the repo-local state names the next useful action for a fresh session, so that a session ending is not the same as the work being lost.

The orchestration ladder

Citadel's architecture document (Gammon 2026a) arranges its capabilities as a ladder, with the instruction to use the cheapest rung that fits:

Table 7: Citadel's orchestration ladder, from the project's architecture document (measured 2026-09-09).

Rung	Duration	Token cost	State kept	Use when
Skill	minutes	low	none	a focused task with a known pattern
Marshal	half an hour to two hours	medium	a session log	several steps, one session
Archon	hours to days	high	a campaign file	work that must survive across sessions
Fleet	days	very high	a session file	three or more parallel streams

A skill is a `SKILL.md` protocol file in the format Agent Skills describes, loaded on demand and costing nothing when not loaded. Marshal chains skills inside one session. Archon runs a multi-session campaign whose state lives in a Markdown campaign file with a feature ledger, a decision log, and a machine-readable continuation point, re-read at the start of every invocation. Fleet spawns agents in isolated git worktrees in waves, and compresses each wave's discoveries to a few hundred tokens so the next wave starts informed rather than rediscovering.

The `/do` router that sits above the ladder narrows a request in four stages. Exact-command matching and a check of active campaign state cost no model tokens; candidate discovery from built-in and project-local skills also costs none; only the final stage, a semantic classifier that reads scope, complexity, persistence, and parallelism, spends around 500 tokens. Only an exact command can skip that classifier, and an explicit route override changes which skill runs without bypassing the approval or verification boundaries.

Enforcement and evidence

The part of Citadel most relevant to the lab's own practice is how much of its policy is enforced rather than advised. Customizing an Agent makes the point that an instruction file is context the model may ignore, and that a rule you cannot afford to have ignored belongs in a hook. Citadel takes that seriously in three layers (Gammon 2026a):

- **Hooks** on every tool call: `PreToolUse` scripts that block edits to protected files and gate pushes and pull requests behind user consent, `PostToolUse` scripts that run the project's type check and file-placement rules, and session-start scripts that scaffold state and restore context after compaction.
- **A spawned judge**: a read-only `policy-enforcer` subagent that receives a proposed hard-to-reverse action and returns a structured allow-or-block verdict, with the most severe rule tier always blocking.
- **Signed telemetry**: every event and artifact record carries a content hash, optionally an `HMAC` signature, and lineage fields linking runs, agents, and tasks.

Verification reports one of four outcomes: passed, failed, blocked, or unknown. Missing evidence is never promoted to success. That is the same instinct as the first rule in Section 19, never to assume an agent's output is correct: an unknown that reads as a pass is worse than a failure you can see.

The project's threat model (Gammon 2026c) is equally plain about what it does not do. Citadel runs with whatever permissions the host runtime has; it is not a sandbox, it does not make an untrusted repository safe to run, and it does not claim to stop prompt injection. Its README says it "does not replace `CLAUDE.md`, `AGENTS.md`, branch protection, or human review" (Gammon 2026d).

What the evidence does and does not show

Citadel is unusual among agent-tooling projects in publishing negative results beside positive ones (Gammon 2026b). Its README states that the first experiment "does not support a savings claim": one baseline timeout drove the aggregate advantage, and removing that pair reversed the economic direction. An outside-authored holdout across 24 repositories verified 3 of 16 tasks for Citadel against 2 of 16 for direct Claude, at similar cost, and the author calls that a diagnostic rather than proof, because a 12.5% baseline is too weak to compare against. The deterministic results are stronger but narrower: journaled recovery produced zero duplicate side effects where a naive restart produced three, and a leased "deploy steward" eliminated every stale-head merge race that independent loops produced across 45 pull requests, under a generated workload in disposable repositories.

The public claim is stated as deliberately narrow: Citadel can make agent evaluations inspectable, reproducible, and honest about failure. Whether it makes a real user faster or cheaper is left open. For a lab deciding whether to adopt it, that candor is itself a reason to trust the rest of the documentation, and also the answer: the case for Citadel is governance and recoverability, not throughput.

Where the lab sits and what the next level would take

The lab's equivalent layer is `Morrison-Lab/ai-config`, described in customizing an agent's worked example. On 2026-09-09 its `main` branch carried:

- 190 skill directories
- 47 hook scripts, each with a test file, registered on three lifecycle events (`UserPromptSubmit`, `PreToolUse`, and `Stop`)
- 8 subagent definitions, including an adversarial reviewer and a prose fact-checker
- 81 shared workflow fragments imported into `CLAUDE.md`
- 73 Python scripts, most of them instruments that a hook or a skill calls

Against Table 7, that configuration sits firmly on the first rung and reaches the others by convention rather than by machinery:

- **Skill.** Fully occupied, and more heavily than Citadel's 48. Nearly every lab convention is a skill, because a skill is the mechanism the model reaches for unprompted.
- **Marshal.** Covered by skills that chain other skills (the loop that drives a pull request through review rounds to a clean verdict is one) and by the harness's own workflow tool. There is no session log; the transcript is the log.
- **Archon.** Partly covered. Persistence across sessions comes from memory files, a per-session lab notebook, and GitHub issues and pull requests as the durable record. Nothing re-reads a campaign file at session start; a new session reconstructs where it was from the issue tracker.
- **Fleet.** Reached in practice, since parallel workers in isolated worktrees are routine, but coordinated by claim comments on issues and by branch naming, not by a claims directory with scope-overlap detection. Two workers editing one file find out at merge time.

The hooks are the sharpest contrast. The lab's 47 are concentrated on three events, and most of them guard the *conversation*: that a reply exists, that a promise ships a mechanism, that a push was reviewed, that a merge was authorized. Citadel's 35 span 29 events and guard the *repository*: protected files, type checks after every edit, file placement, consent before a push. Both are enforcement; they enforce different things.

So the next rung, if the lab wants it, is not more skills. Concretely it would take:

- **A session-start hook** that scaffolds and re-reads resumable state, so a fresh session begins from a named next action rather than from a tracker search. This is Citadel's "Resume" state, and the lab notebook already holds the content; what is missing is the hook that loads it.
- **`PostToolUse` quality gates** that run the project's own checks (a spell check, a lint, a render of the edited page) after each edit instead of at push time.
- **File-level claims for parallel workers**, so that two sessions editing the same fragment collide at claim time rather than at merge time.
- **A verification vocabulary with an explicit "unknown"**, so that a check that did not run is reported as not having run.

None of that requires adopting Citadel. Installing it would add 2,031 files of plugin, four state directories to every repository it manages, and a single-maintainer dependency, in exchange

for machinery the lab has mostly rebuilt in its own idiom. The more useful reading of Citadel is as a reference design: a worked answer to what each rung of the ladder costs and what state it needs, written by someone who then measured whether it helped and published the answer either way.

21 Firewall and Network Configuration

Coding agents require specific network access to function properly. If a coding agent is running behind a corporate firewall or on a restricted network, you may need to configure allowlists to enable coding agent functionality.

Built-in Agent Firewall

Coding agents run in a GitHub Actions environment with a built-in firewall that limits internet access by default. This firewall helps protect against:

- Data exfiltration
- Accidental leaks of sensitive information
- Execution of malicious instructions

By default, the agent's firewall allows access to:

- Common OS package repositories (Debian, Ubuntu, Red Hat, etc.)
- Popular container registries (Docker Hub, Azure Container Registry, AWS ECR, etc.)
- Language-specific package registries (npm, PyPI, Maven, RubyGems, etc.)
- Common certificate authorities for SSL validation

For the complete list of allowed hosts, see the Copilot allowlist reference.

Customizing Agent Firewall Settings

In your repository's "Coding agent" settings page, you can:

- Add custom hosts to the allowlist (for internal dependencies or additional registries)
- Opt out of the default recommended allowlist for stricter security
- Disable the firewall entirely (not recommended)

If a coding agent's request is blocked by the firewall, a warning will be added to the pull request or comment, detailing the blocked address and the command that triggered it.

For more information, see Customizing or disabling the firewall for GitHub Copilot coding agent.

Recommended URLs for Data Science Repositories

The lab maintains its working allowlists in Morrison-Lab/stats-allowlist (Morrison Lab 2026b):

- `allowlist.txt` for the Copilot coding agent
- `claude-allowlist.txt` for Claude Code cloud sessions (see Configuring network access for Claude cloud sessions)

Treat those files as the source of truth. When an agent's proxy refuses a host it needs:

1. Add the host to the matching file.
2. Apply the updated list to the agent itself: the repository's "Coding agent" settings page for Copilot, or the cloud environment for Claude. Editing the file alone does not change what either agent can reach.

The list below gives examples of useful hosts and what each one serves. It is not kept in sync with `allowlist.txt`, so check that file for the current entries.

For data science and R-focused repositories, we recommend adding the following URLs to your Copilot allowlist. These sites are reputable sources of documentation and packages that coding agents may need to access:

R Package Documentation and Ecosystems:

- `tidyverse.org` - `{tidyverse}` package documentation and learning resources
- `r-lib.org` - Core R infrastructure packages (`{devtools}`, `{testthat}`, `{usethis}`, etc.)
- `ggplot2.tidyverse.org` - `{ggplot2}` visualization package
- `dplyr.tidyverse.org` - `{dplyr}` data manipulation package
- `tidyr.tidyverse.org` - `{tidyr}` data tidying package
- `purrr.tidyverse.org` - `{purrr}` functional programming package
- `readr.tidyverse.org` - `{readr}` data reading package
- `stringr.tidyverse.org` - `{stringr}` string manipulation package
- `forcats.tidyverse.org` - `{forcats}` categorical data package

R Package Repositories:

- `cran.r-project.org` - The Comprehensive R Archive Network
- `cloud.r-project.org` - CRAN mirror (cloud-based)
- `docs.ropensci.org` - rOpenSci package documentation (e.g., `{targets}`)
- `rdatatable.gitlab.io` - `{data.table}` package documentation
- `rstudio.github.io` - RStudio-maintained packages (e.g., `{renv}`)

Code Style and Quality Tools:

- `styler.r-lib.org` - `{styler}` code formatting package
- `lintr.r-lib.org` - `{lintr}` code linting package

- `roxygen2.r-lib.org` - {roxygen2} documentation package
- `style.tidyverse.org` - Tidyverse style guide

General Documentation and Reference:

- `en.wikipedia.org` - General reference and technical documentation
- `r-project.org` - Official R project website
- `quarto.org` - Quarto publishing system documentation
- `pandoc.org` - Pandoc document converter documentation

GitHub Organizations (for package repositories):

- `https://github.com/tidyverse/` - Tidyverse package source code
- `https://github.com/r-lib/` - R-lib package source code
- `https://github.com/rstudio/` - RStudio package source code
- `https://github.com/ropensci/` - rOpenSci package source code

When to Add These URLs

Add these URLs to your repository's allowlist if:

- Coding agents report blocked access to these sites
- You're working on R or data science projects that use these packages
- You want agents to access current documentation during code generation

You can add URLs selectively based on your project's specific dependencies rather than adding all URLs at once.

Safety of These URLs

The hosts listed here are reputable documentation and package sources:

- Most are run by established R projects and organizations (Tidyverse, RStudio/Posit, R Core Team, rOpenSci)
- They are widely used in the R community
- They focus on documentation and package distribution

Reputable does not mean that every page on a host is vetted. Every allowed host is a place the agent can send data to, or read instructions from (Morrison Lab 2026a). Claude's allowlist takes one domain per line (Anthropic 2026a). A path entry such as `https://github.com/tidyverse/` cannot be expressed there; allowing GitHub means allowing all of `github.com`. Copilot accepts such a URL and limits it to that path and the paths below it (GitHub 2026a), so an entry for one GitHub organization still covers that organization's issues and pull requests, including comments from other users. Any

> GitHub user or organization can publish a site at `<owner>.github.io` (GitHub 2026b), so a `*.github.io` entry opens sites that anyone can publish, while `rstudio.github.io` serves only the `rstudio` organization's sites. Anyone can edit Wikipedia (Wikipedia contributors 2026). These hosts are often needed for package installs and documentation; allow them knowingly (see Configuring network access for Claude cloud sessions).

References

2001: A Space Odyssey. 1968. Film. https://en.wikipedia.org/wiki/2001:_A_Space_Odyssey_(film).

Akhouri, Udit. 2026. *Quicksilver: Claude Code Skill and Plugin for Bulk Judgment Delegation*. GitHub repository. https://github.com/UditAkhourii/quicksilver.

Amazon Web Services. 2026. *Kiro Documentation*. Documentation. https://kiro.dev/docs/.

Anthropic. 2026a. *Configure Cloud Environments*. Documentation. https://code.claude.com/docs/en/cloud-environments.

Anthropic. 2026b. *Use Research on Claude*. Help Center. https://support.claude.com/en/articles/11088861-use-research-on-claude.

Anthropic. 2026c. *Web Search Tool*. Documentation. https://platform.claude.com/docs/en/agents-and-tools/tool-use/web-search-tool.

Anthropic. 2026d. *When Should i Use Web Search, Extended Thinking, and Research?* Help Center. https://support.claude.com/en/articles/11095361-when-should-i-use-web-search-extended-thinking-and-research.

Asimov, Isaac. 1950. *I, Robot*. Novel; Gnome Press. https://search.library.ucdavis.edu/permalink/01UCD_INST/9fle3i/alma990000226350403126.

Battlestar Galactica. 2004. Television Series. https://en.wikipedia.org/wiki/Battlestar_Galactica_(2004_TV_series).

Blade Runner. 1982. Film. https://en.wikipedia.org/wiki/Blade_Runner.

Card, Orson Scott. 1985. *Ender's Game*. Novel; Tor Books. https://en.wikipedia.org/wiki/Ender%27s_Game.

Cline. 2026. *Cline: AI Coding, Open Source and Open Choice*. Product page. https://cline.bot/.

Cognition. 2026. *Introducing Devin*. Documentation. https://docs.devin.ai/get-started/devin-intro.

Cursor. 2026. *Cloud Agents*. Documentation. https://cursor.com/docs/cloud-agent.

Flight of the Conchords. 2007. *The Humans Are Dead*. Music Video. https://www.youtube.com/watch?v=B1BdQcJ2ZYY.

Gammon, Seth. 2026a. *Citadel Architecture*. Documentation. https://github.com/SethGammon/Citadel/blob/main/docs/ARCHITECTURE.md.

Gammon, Seth. 2026b. *Citadel Proof Experiments*. Documentation. https://github.com/SethGammon/Citadel/blob/main/docs/EXPERIMENTS.md.

Gammon, Seth. 2026c. *Citadel Threat Model*. Documentation. https://github.com/SethGammon/Citadel/blob/main/THREAT_MODEL.md.

Gammon, Seth. 2026d. *Citadel: An Open-Source Operating Layer for Claude Code and OpenAI Codex*. GitHub repository. https://github.com/SethGammon/Citadel.

GitHub. 2026a. *Customizing or Disabling the Firewall for GitHub Copilot Coding Agent*. Documentation. https://docs.github.com/en/copilot/how-tos/use-copilot-agents/coding-agent/customize-the-agent-firewall.

GitHub. 2026b. *What Is GitHub Pages?* Documentation. https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages.

Google. 2026a. *Deep Research Max: A Step Change for Autonomous Research Agents*. The Keyword (blog). https://blog.google/innovation-and-ai/models-and-research/gemini-models/next-generation-gemini-deep-research/.

Google. 2026b. *Gemini Apps Limits and Upgrades for Google AI Subscribers*. Gemini Apps Help. https://support.google.com/gemini/answer/16275805.

Google. 2026c. *Gemini Deep Research Agent*. Gemini API documentation. https://ai.google.dev/gemini-api/docs/deep-research.

Google. 2026d. *Getting Started with Jules*. Documentation. https://jules.google.com/docs/.

Google. 2026e. *Use Deep Research in Gemini Apps*. Gemini Apps Help. https://support.

google.com/gemini/answer/15719111.

Herbert, Frank. 1965. *Dune*. Novel; Chilton Books. https://en.wikipedia.org/wiki/Organizations_of_the_Dune_universe#Thinking_machines.

Hyperautomation Labs. 2026. *Never Hit a Claude Limit Again — 12 Fixes Ranked by How Much Window They Buy*. Video. https://www.youtube.com/watch?v=dIP-4Mc6ZfA.

LeCun, Yann. 2022. *A Path Towards Autonomous Machine Intelligence*. Meta AI Research; New York University; Technical Report. https://openreview.net/forum?id=BZ5a1r-kVsf.

Microsoft. 2026a. *Conversation Modes in Microsoft Copilot*. Microsoft Support. https://support.microsoft.com/en-us/microsoft-copilot/conversation-modes-in-microsoft-copilot.

Microsoft. 2026b. *Deep Research in Microsoft Copilot*. Microsoft Support. https://support.microsoft.com/en-us/microsoft-copilot/deep-research-in-microsoft-copilot.

Microsoft. 2026c. *Get Started with Researcher in Microsoft 365 Copilot*. Microsoft Support. https://support.microsoft.com/en-us/microsoft-365-copilot/get-started-with-researcher-in-microsoft-365-copilot.

Microsoft. 2026d. *Microsoft Copilot Researcher Agent Frequently Asked Questions*. Microsoft Learn. https://learn.microsoft.com/en-us/copilot/microsoft-365/faq-researcher.

Microsoft. 2026e. *What Is Researcher Agent in Microsoft Copilot?* Microsoft Learn. https://learn.microsoft.com/en-us/microsoft-365/copilot/researcher-agent.

Morrison Lab. 2026a. *Merge Additional Hosts into Claude Allowlist; Prune Redundant and Risky Entries*. GitHub pull request. https://github.com/Morrison-Lab/stats-allowlist/pull/6.

Morrison Lab. 2026b. *Stats-Allowlist: Network Allowlists for AI Coding Agents*. GitHub repository. https://github.com/Morrison-Lab/stats-allowlist.

Ollama. 2026. *Ollama*. Product page. https://ollama.com/.

OpenAI. 2026a. *Deep Research*. OpenAI API documentation. https://developers.openai.com/api/docs/guides/deep-research.

OpenAI. 2026b. *Deep Research in ChatGPT*. Help Center. https://help.openai.com/en/articles/10500283-deep-research-faq.

OpenCode. 2026. *Intro: AI Coding Agent Built for the Terminal*. Documentation. https:

//opencode.ai/docs/.

OpenHands. 2026. *OpenHands Documentation: Introduction*. Documentation. https://docs.openhands.dev/.

Perplexity. 2026a. *Sonar Deep Research*. Perplexity API documentation. https://docs.perplexity.ai/docs/sonar/models/sonar-deep-research.

Perplexity. 2026b. *What Is Research Mode?* Help Center. https://www.perplexity.ai/help-center/en/articles/10738684-what-is-research-mode.

Perplexity. 2026c. *What's New in Advanced Deep Research*. Help Center. https://www.perplexity.ai/help-center/en/articles/13600190-what-s-new-in-advanced-deep-research.

Stanford Online, and Andrew Ng. 2026. *Andrew Ng: One Skill to Stay Relevant in the Age of AI*. Video. https://www.youtube.com/watch?v=HTlf3KNNez0.

Terminator 3: Rise of the Machines. 2003. Film. https://en.wikipedia.org/wiki/Terminator_3:_Rise_of_the_Machines.

The Matrix. 1999. Film. https://en.wikipedia.org/wiki/The_Matrix.

WarGames. 1983. Film. https://en.wikipedia.org/wiki/WarGames.

Warp. 2026. *Getting Started with Warp*. Documentation. https://docs.warp.dev/.

Wikipedia contributors. 2026. *Wikipedia:about*. Wikipedia. https://en.wikipedia.org/wiki/Wikipedia:About.